1. Application UI

The React frontend is deployed at https://myapp.local and served through the NGINX Ingress Controller with TLS termination via cert-manager. The browser shows 'Not secure' because a self-signed certificate is used (expected for local development — production would use Let's Encrypt). The form accepts a Name and Email and POSTs to /api/register on the backend. Registered users are fetched from /api/users and displayed as a list below the form.

Screenshot 1.1 — Live Application (https://myapp.local)

URL: https://myapp.local | TLS: cert-manager self-signed certificate

User 'prem - premsaikoredla@gmail.com' has been successfully registered and is displayed.

Frontend (React + Nginx) → Backend (Node.js :5000) → MongoDB — full stack working end-to-end.

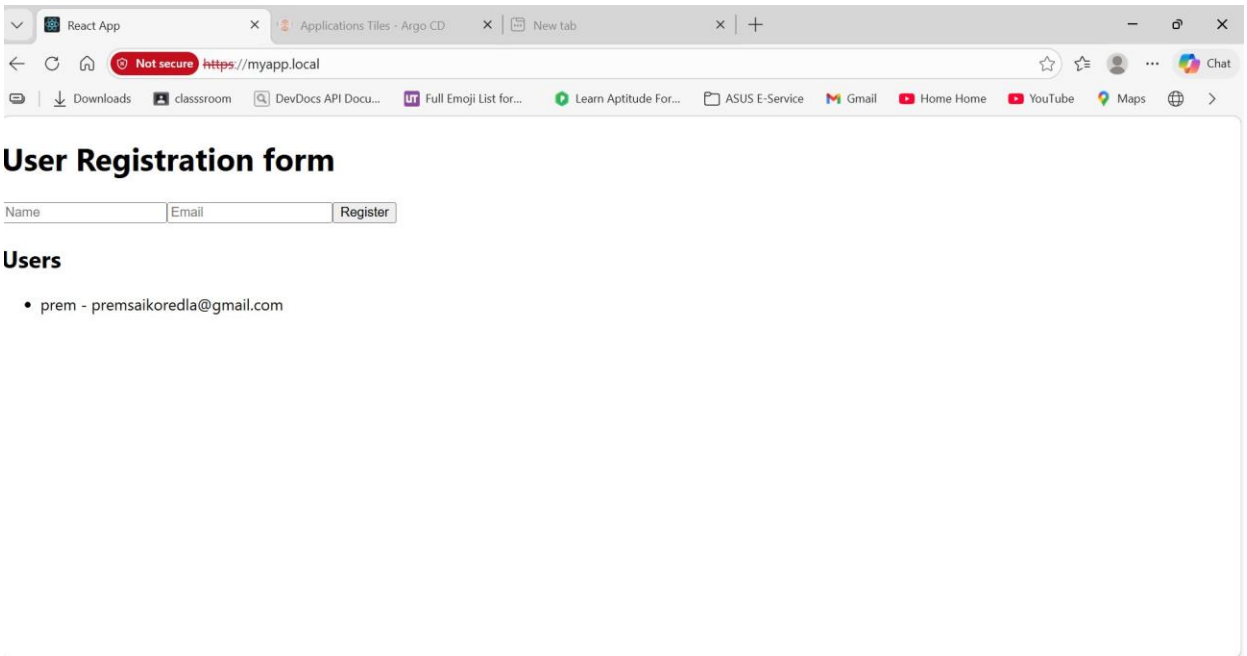

Figure 1.1 — User Registration app running at https://myapp.local with a registered user visible

2. Kubernetes Cluster — Pod Status

The following screenshots show the pod lifecycle from initial deployment to fully running state. kubectl get pods was run twice to capture both the ContainerCreating phase (immediately after ArgoCD sync) and the Running phase (once all images pulled and containers started). All pods run in the default namespace.

2.1 Pods — ContainerCreating (Initial Deployment)

Screenshot 2.1 — Pods spinning up immediately after sync

2 backend pods (0/2 Ready) — waiting for both app container + jaeger-agent sidecar to start.

2 frontend pods (0/1 Ready) — nginx container starting up.

1 mongodb-0 StatefulSet pod (0/1 Ready) — PVC binding and container initialization.

All pods at AGE: 20s — captured moments after ArgoCD applied the manifests.

```
prems@LAPTOP-DV8V9H70 MINGW64 ~/Downloads/dodo_payments/task1-app (main)
$ kubectl get pods
NAME                        READY   STATUS              RESTARTS   AGE
backend-bfc5659f-n5xw7      0/2     ContainerCreating   0          20s
backend-bfc5659f-qhp52      0/2     ContainerCreating   0          20s
frontend-69fc6c7d9c-27md5   0/1     ContainerCreating   0          20s
frontend-69fc6c7d9c-n8t6v   0/1     ContainerCreating   0          20s
mongodb-0                   0/1     ContainerCreating   0          20s

prems@LAPTOP-DV8V9H70 MINGW64 ~/Downloads/dodo_payments/task1-app (main)
$
```

Figure 2.1 — All pods in ContainerCreating state (20s after deployment)

2.2 Pods — Running (Healthy State)

Screenshot 2.2 — All pods fully Running after 11 minutes

backend pods: 2/2 Running — both the Express app container AND the jaeger-agent sidecar are up.

frontend pods: 1/1 Running — Nginx serving the React SPA.

mongodb-0: 1/1 Running — StatefulSet with PVC 1Gi bound and MongoDB accepting connections.

RESTARTS: 0 across all pods — clean startup, no crash loops.

```
prems@LAPTOP-DV8V9H70 MINGW64 ~/Downloads/dodo_payments/task1-app (main)
$ kubectl get pods
NAME                          READY   STATUS    RESTARTS   AGE
backend-bfc5659f-n5xw7        2/2     Running   0          11m
backend-bfc5659f-qhp52        2/2     Running   0          11m
frontend-69fc6c7d9c-27md5     1/1     Running   0          11m
frontend-69fc6c7d9c-n8t6v     1/1     Running   0          11m
mongodb-0                     1/1     Running   0          11m

prems@LAPTOP-DV8V9H70 MINGW64 ~/Downloads/dodo_payments/task1-app (main)
```

Figure 2.2 — All pods Running with 0 restarts (11 minutes after deployment)

3. ArgoCD — GitOps Continuous Delivery

ArgoCD (v3.3.2) is used for GitOps-based continuous delivery. GitHub Actions CI/CD pipeline builds and pushes Docker images tagged with the Git commit SHA, then commits an image update to the repository. ArgoCD auto-syncs the manifests/ directory from the repository and applies changes to the cluster. Five ArgoCD applications manage the full stack.

3.1 Application Sync Status — Main App

Screenshot 3.1 — 'task' application: Healthy & Synced

App Name: task | Project: default | Status: Healthy + Synced

Repository: https://github.com/premsaieswarnaidu/app_1.git | Path: manifests

Target Revision: HEAD | Destination: in-cluster | Namespace: default

Auto sync is enabled — ArgoCD watches the repo and applies changes automatically.

Last Sync: 02/28/2026 13:02:56 — synced from GitHub Actions commit 'Update images to cdd8c73ef18...'

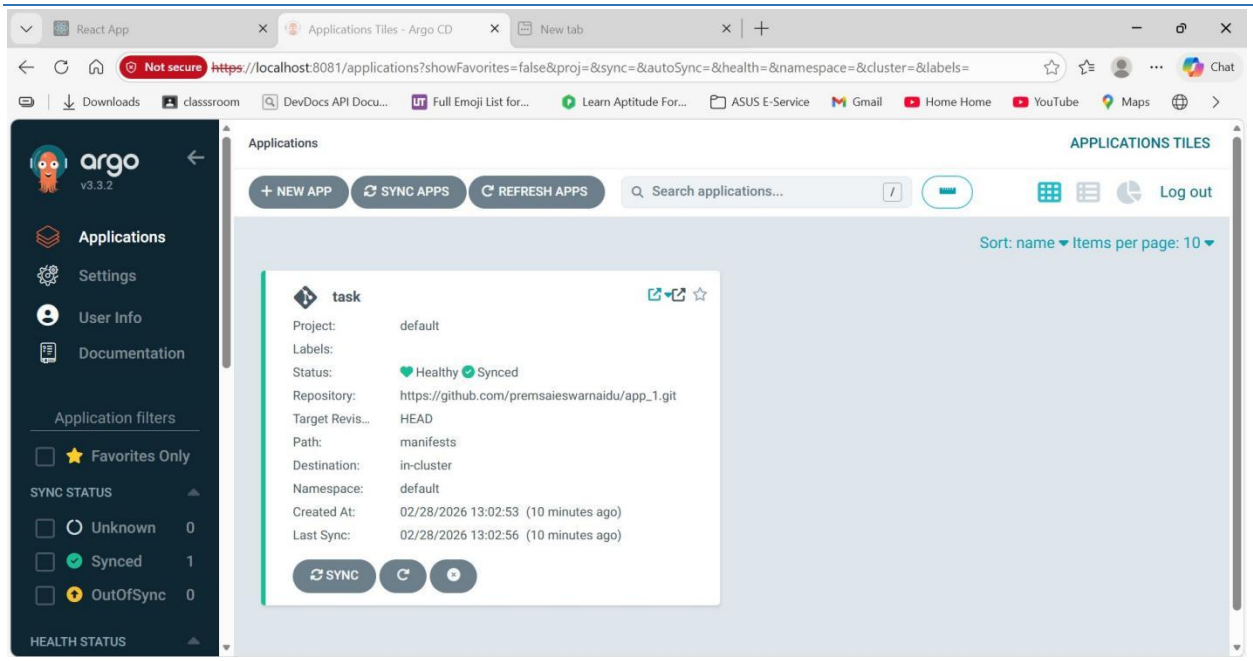

Figure 3.1 — ArgoCD 'task' application: Healthy, Synced to HEAD (4f7ff47)

3.2 Application Details Tree

Screenshot 3.2 — ArgoCD Resource Tree showing deployed Kubernetes objects

App Health: Progressing (pods starting up at time of screenshot).

Sync Status: Synced to HEAD (4f7ff47) — auto sync enabled.

Last Sync OK: Succeeded at 13:02:56 — author: github-actions.

Commit message confirms GitOps flow: 'Update images to cdd8c73ef18e29d77b762...'

Resource tree shows: backend-config (ConfigMap), backend (Service) and all other manifests.

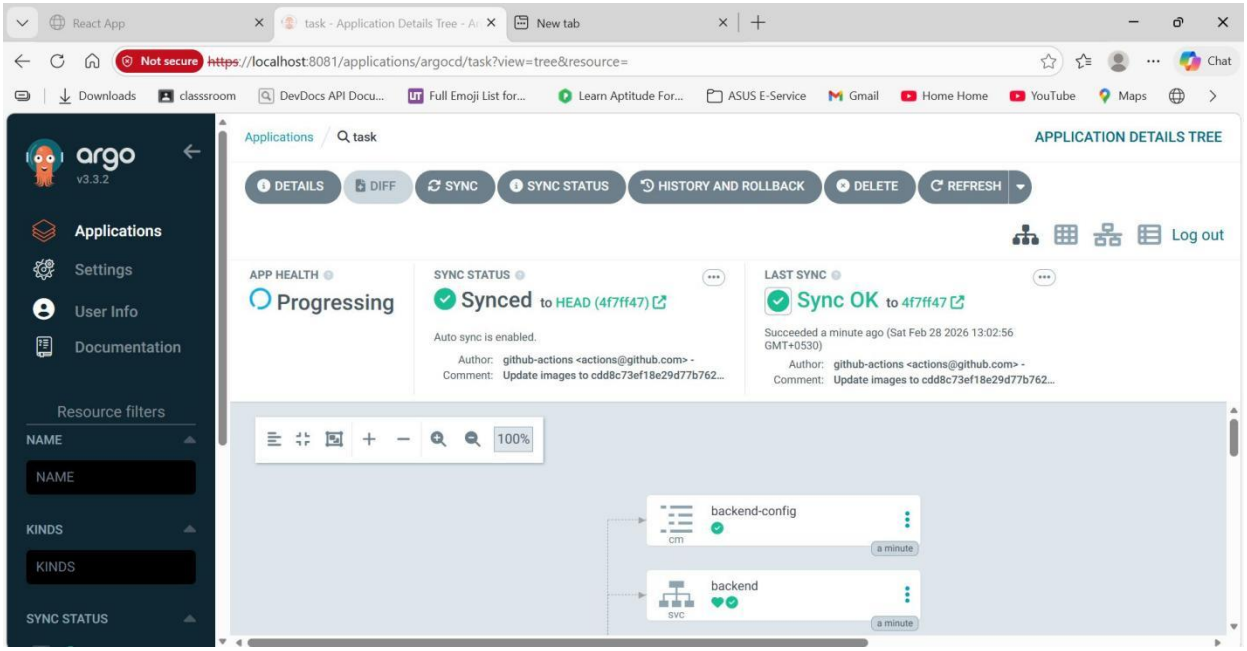

Figure 3.2 — ArgoCD Application Details Tree for 'task' app

3.3 Observability Applications in ArgoCD

Screenshot 3.3 — All observability stack applications managed by ArgoCD

efk application: Healthy + Synced | Path: observability/efk | Namespace: observability

grafana application: Healthy + Synced | Path: observability/grafana | Namespace: observability

jaeger application: (visible below) Healthy + Synced | Path: observability/jaeger

prometheus application: (visible below) Healthy + Synced | Path: observability/prometheus

Total: 5 Synced applications managing the full app_1 stack via GitOps from a single repository.

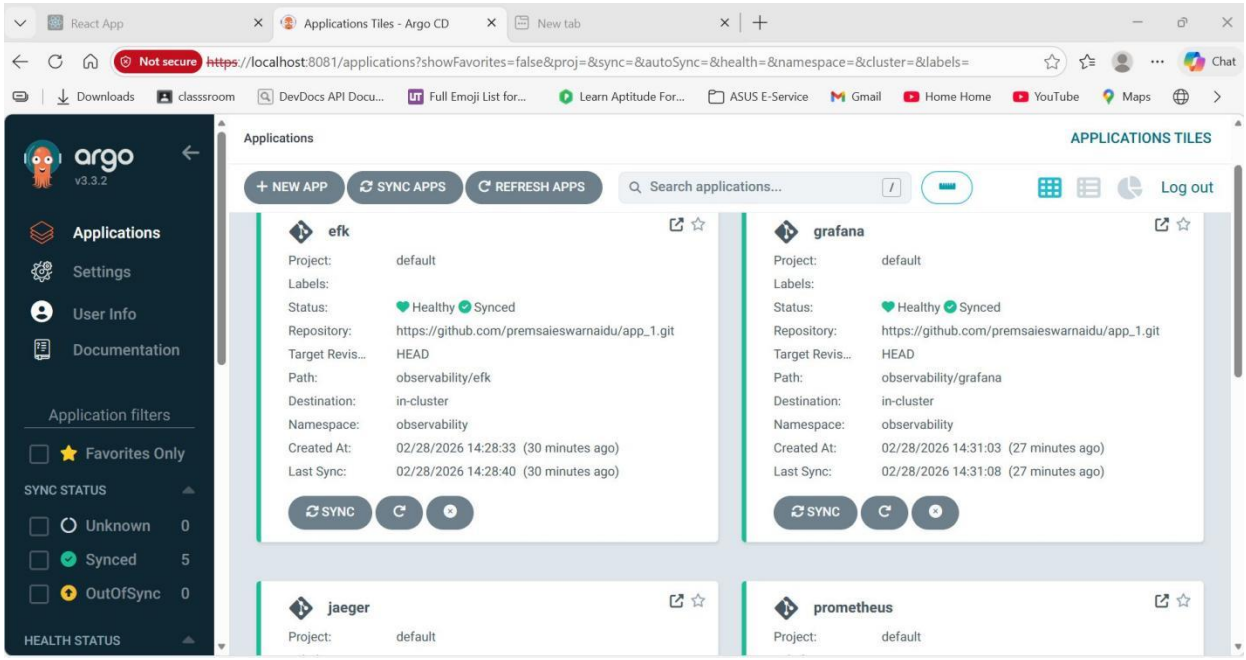

Figure 3.3 — ArgoCD Applications dashboard: EFK, Grafana, Jaeger, Prometheus all Healthy & Synced

4. Prometheus — Metrics Collection & Alerting

Prometheus is deployed in the observability namespace and scrapes metrics from the backend service at http://backend.default.svc.cluster.local:5000/metrics. The backend exposes a custom Prometheus histogram metric (http_request_duration_seconds) via the prom-client Node.js library, plus default Node.js runtime metrics. Prometheus also monitors itself.

4.1 Prometheus Scrape Targets

> **Screenshot 4.1 — Prometheus Targets page showing both scrape jobs UP**
>
> backend (1/1 up): Endpoint http://backend.default.svc.cluster.local:5000/metrics — State: UP
>
> Last Scrape: 10.186s ago | Scrape Duration: 125.356ms | No errors
>
> Labels: instance='backend.default.svc.cluster.local:5000', job='backend'
>
> prometheus (1/1 up): Self-scrape at http://localhost:9090/metrics — State: UP
>
> All targets healthy — Prometheus is successfully collecting metrics from the backend.

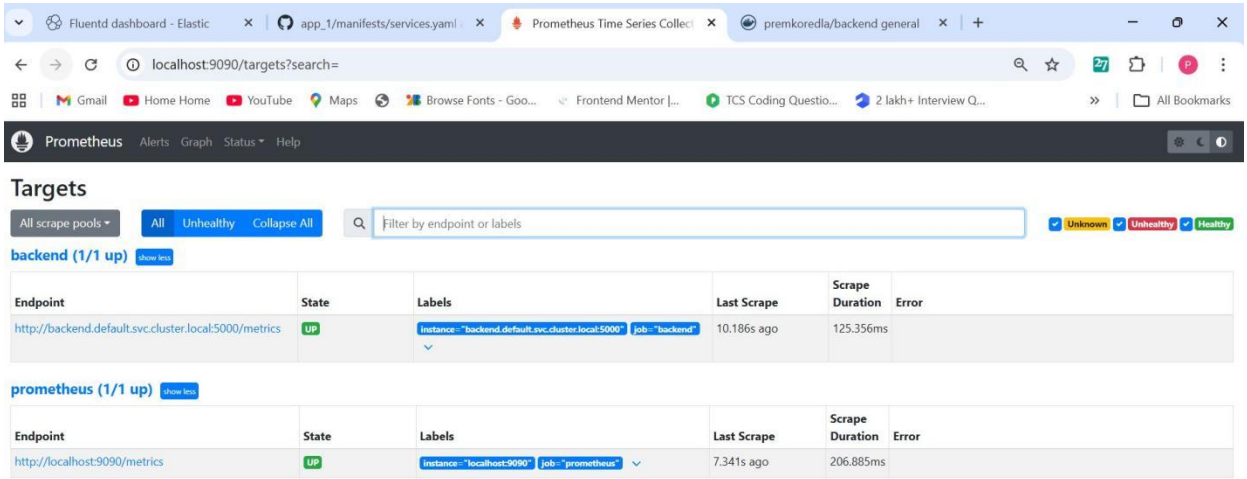

Figure 4.1 — Prometheus Targets: backend (1/1 UP) and prometheus (1/1 UP)

4.2 Prometheus Query — Request Rate by Route

> **Screenshot 4.2 — PromQL query: sum(rate(http_request_duration_seconds_count[1m])) by (route)**
>
> Query executed: sum(rate(http_request_duration_seconds_count[1m])) by (route)
>
> Results show request rate per route:
>
> {route='/health'}: 0.8445 req/s — Kubernetes liveness probe polling every ~30s
>
> {route='/ready'}: 0.8445 req/s — Kubernetes readiness probe polling every ~10s
>
> {route='/metrics'}: 0.3111 req/s — Prometheus scrape interval (every ~15s)
>
> Load time: 94ms | Result series: 3 — metric is working correctly from the backend.

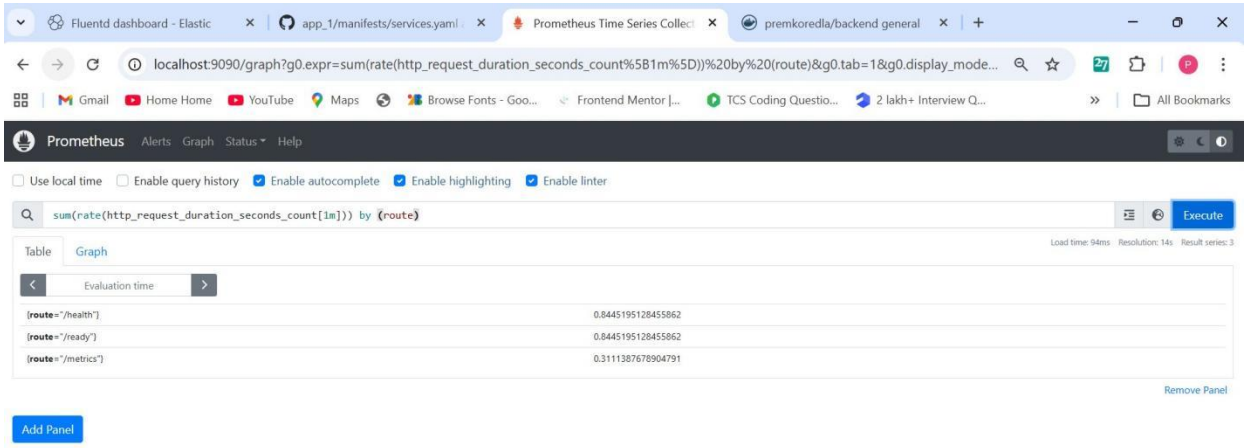

Figure 4.2 — Prometheus Graph: request rate by route, showing probe and scrape traffic

4.3 Prometheus Alert Rules

> **Screenshot 4.3 — HighErrorRate alert rule configured and Inactive (healthy system)**
>
> Alert Name: HighErrorRate | Group: app-alerts | Severity: critical
>
> Expression: rate(http_request_duration_seconds_count{status_code=~'5..'}[5m]) > 0.05
>
> For: 2 minutes | Status: INACTIVE (0 active) — no 5xx errors detected, system healthy.
>
> Source file: /etc/prometheus/rules/alert-rules.yaml — loaded from the alert-rules ConfigMap.
>
> This alert fires when more than 5% of requests return 5xx errors for 2 consecutive minutes.

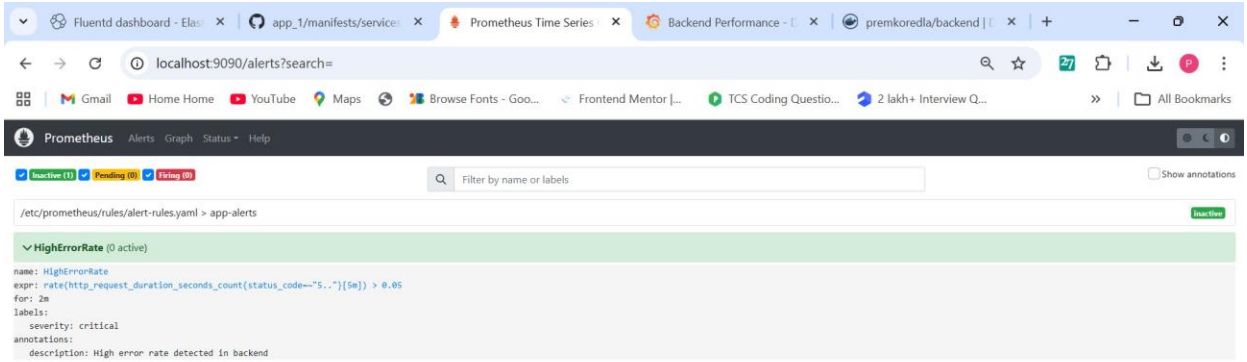

Figure 4.3 — Prometheus Alerts: HighErrorRate rule loaded and INACTIVE (system healthy)

5. Grafana — Backend Performance Dashboard

Grafana (v10.4.0) is deployed in the observability namespace with a pre-configured datasource pointing to Prometheus, and a custom 'Backend Performance' dashboard provisioned via ConfigMap. The dashboard visualises four key panels from real-time Prometheus data scraped from the backend's /metrics endpoint.

5.1 Backend Performance Dashboard

> **Screenshot 5.1 — Grafana Backend Performance dashboard (last 5 minutes)**
>
> Panel 1 — Request Rate: Shows traffic per route. Peak ~10 req/s on /ready probe.
>
> At 14:41: /api/users: 0, /health: 0.0889, /metrics: 0.0667, /ready: 0.0889 req/s
>
> Panel 2 — Average Request Duration: All routes consistently under 0.05s — healthy response times.
>
> Panel 3 — Latency (p95): /api/users p95 at ~0.2s — acceptable for a MongoDB query route.
>
> Panel 4 — Error Rate (%): 'No data' — confirms zero 5xx errors during the observation window.
>
> Data source: backend.default.svc.cluster.local:5000 | job=backend

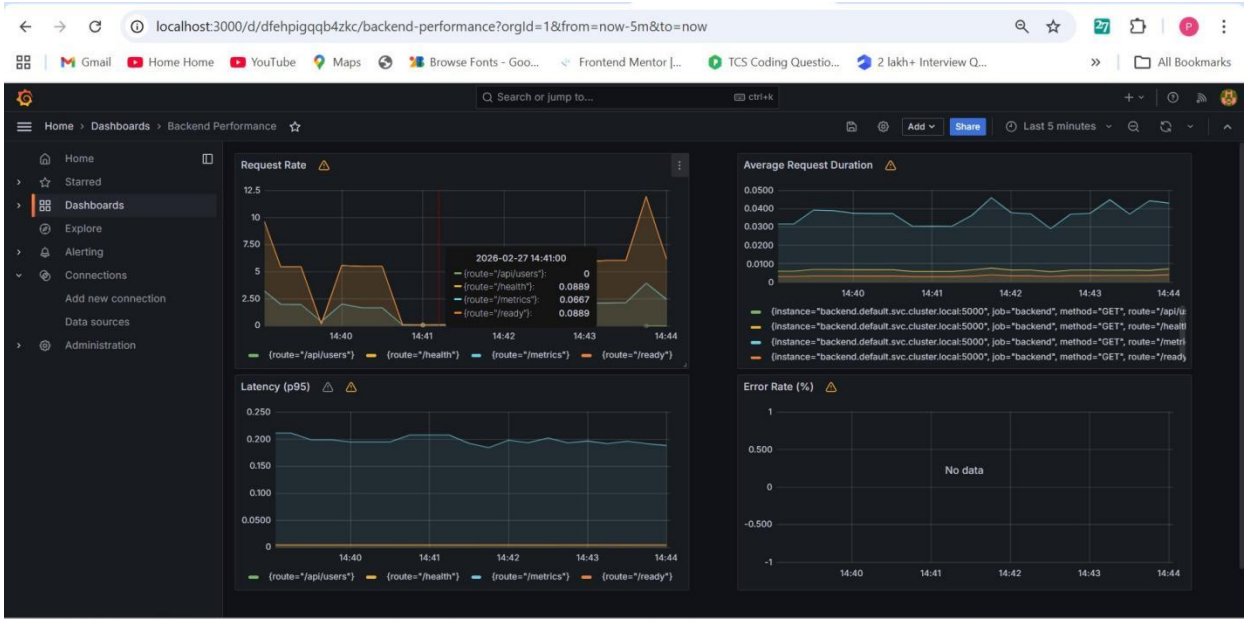

Figure 5.1 — Grafana Backend Performance dashboard: Request Rate, Latency, p95, Error Rate

6. Jaeger — Distributed Tracing

Jaeger (all-in-one:1.55) is deployed in the observability namespace. A Jaeger Agent sidecar container runs inside each backend pod, listening on UDP port 6831. Application traces are batched by the agent and forwarded to the Jaeger Collector at jaeger.observability.svc.cluster.local:14250 via gRPC. The Jaeger UI is available at localhost:16686.

6.1 Jaeger Trace Search

> **Screenshot 6.1 — Jaeger UI showing 20 traces from the last hour**
>
> Service: jaeger-all-in-one (the Jaeger service itself is traced as a demo).
>
> Operations: 3 operations visible | Lookback: Last Hour | 20 traces found.
>
> Trace examples:
>
> /api/services/{service}/operations — 145.14ms (1 span)
>
> /api/services — 164.15ms (1 span)
>
> /api/services/{service}/operations — 11.56ms (1 span, faster cached call)
>
> Scatter plot (top) shows trace duration over time — no outliers or latency spikes.
>
> The jaeger-agent sidecar in the backend pod forwards traces to the Jaeger Collector gRPC endpoint.

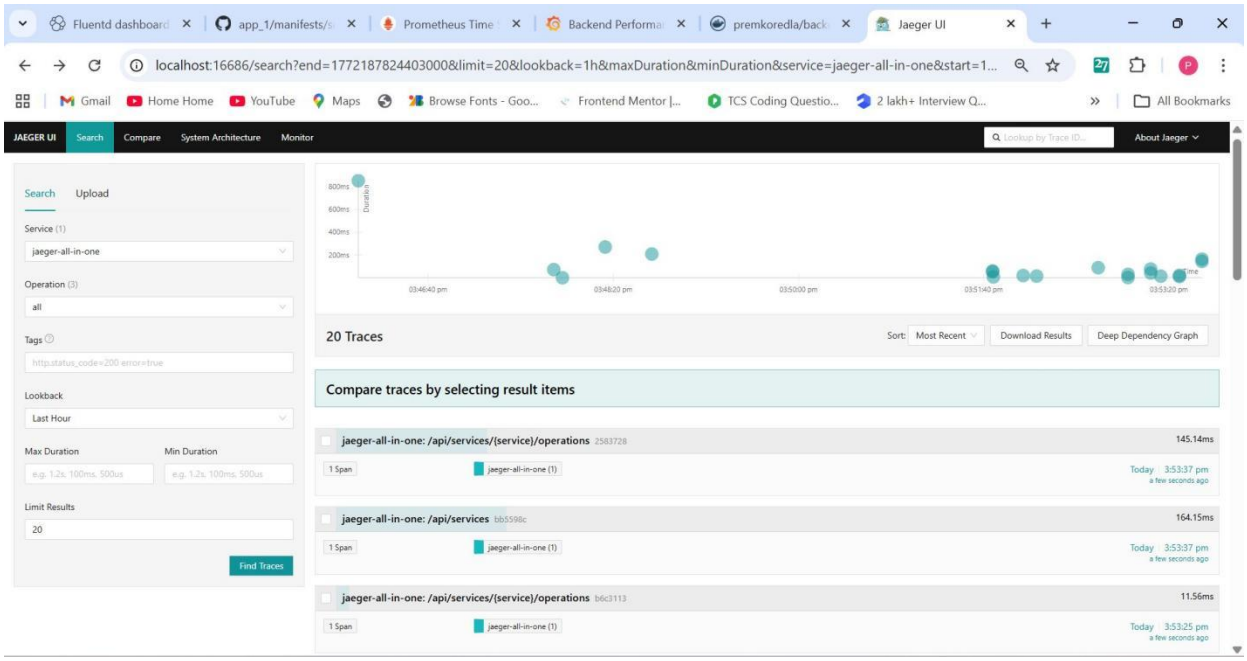

Figure 6.1 — Jaeger UI: 20 traces found, showing trace duration scatter plot and trace list

7. Kibana — Log Aggregation (EFK Stack)

The EFK stack (Elasticsearch + Fluentd + Kibana) is deployed in the observability namespace and managed by ArgoCD. Fluentd runs as a DaemonSet on every Kubernetes node, collecting stdout logs from all pods and shipping them to Elasticsearch indexed as 'fluentd-YYYY.MM.DD'. Kibana provides search and visualisation at localhost:5601.

7.1 Kibana Discover — Log Stream

> **Screenshot 7.1 — Kibana Discover view: fluentd-logs index, 2,410 hits (last 15 minutes)**
>
> Index pattern: fluentd-logs | Time range: Last 15 minutes
>
> 2,410 log entries collected — Fluentd DaemonSet successfully shipping pod logs to Elasticsearch.
>
> Available fields: @timestamp, message, tag | Interval: 30 seconds auto-binning.
>
> Log entries show Kubernetes system and ArgoCD controller logs with full JSON payloads.
>
> Example log: ArgoCD notifications controller processing 'argocd/task1' resource.
>
> Tag format: kubernetes.var.log.containers.<pod>_<namespace>_<container>-<id>.log

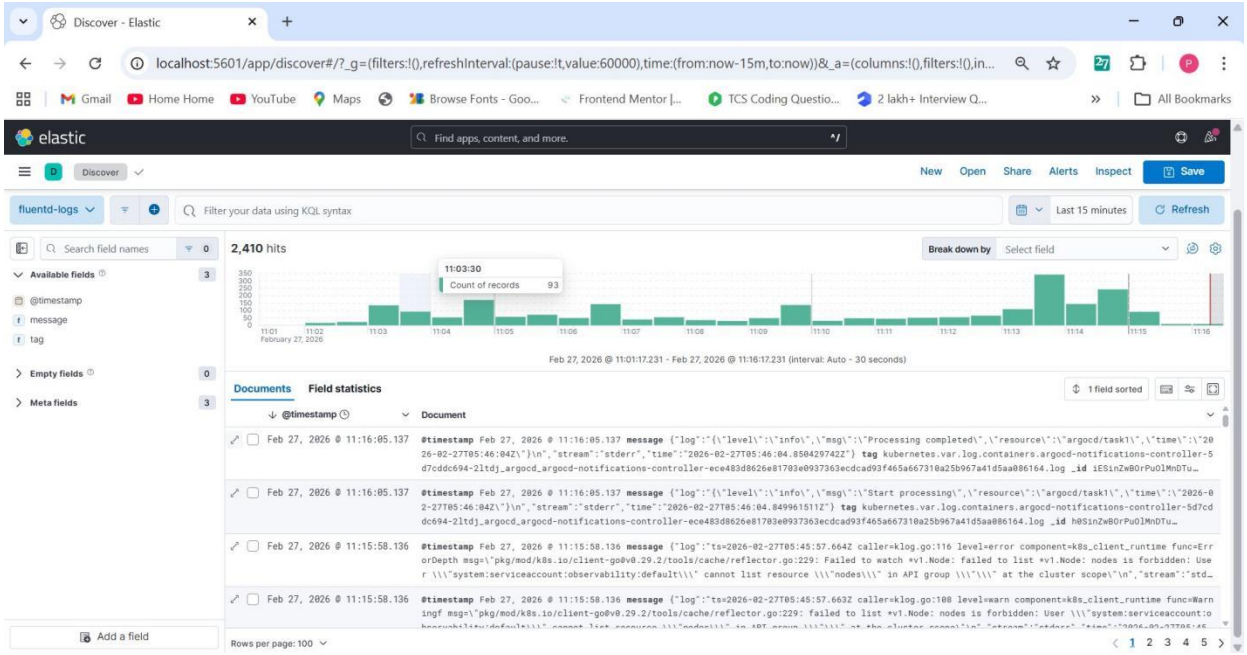

Figure 7.1 — Kibana Discover: 2,410 log hits from fluentd-logs index (last 15 minutes)

7.2 Kibana — Expanded Log Document

> **Screenshot 7.2 — Expanded single log document showing Fluentd log structure**
>
> Index: fluentd-2026.02.27 | Document 2 of 500 in current result set.
>
> Fields in each document:
>
> _id: XESlnZwBOrPuOlMneD6a (Elasticsearch document ID)
>
> @timestamp: Feb 27, 2026 @ 11:19:48.122 (nanosecond precision)
>
> message: JSON log payload — Elasticsearch DNS query log from kube-system/coredns
>
> tag: kubernetes.var.log.containers.coredns-<id>_kube-system_coredns-<hash>.log
>
> This confirms Fluentd is collecting logs from ALL namespaces including kube-system.

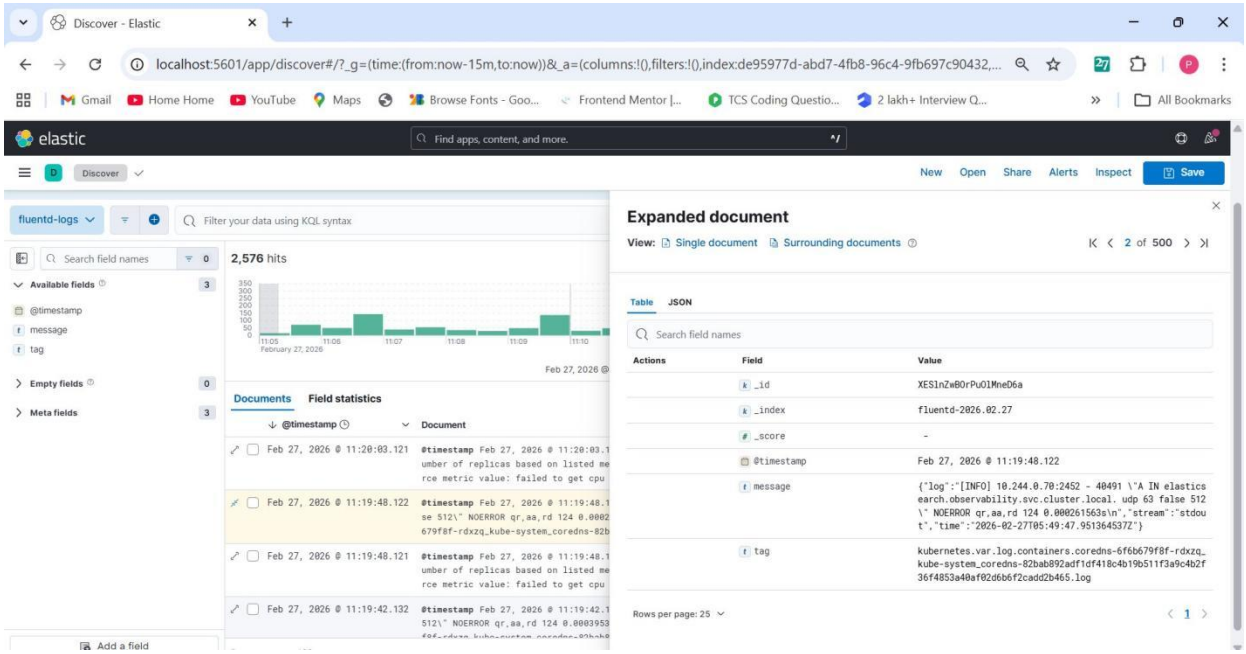

Figure 7.2 — Kibana Discover: Expanded document showing fluentd-2026.02.27 log structure

7.3 Kibana — Fluentd Log Volume Dashboard

> **Screenshot 7.3 — Kibana custom dashboard: 'Log Volume Over Time'**
> Dashboard: Fluentd dashboard | Time range: Last 15 minutes (11:21–11:35, Feb 27 2026)
> Panel: Log Volume Over Time — bar chart, @timestamp per 30 seconds on X-axis.
> Peak at 11:21: ~148 records/30s — high activity at pod startup / initial log flush.
> Steady state: 25–70 records/30s — consistent log collection from all running pods.
> Dashboard was built using 'Create visualization' in Kibana with the fluentd-logs data view.
> This confirms the full EFK pipeline: Fluentd → Elasticsearch → Kibana visualisation.

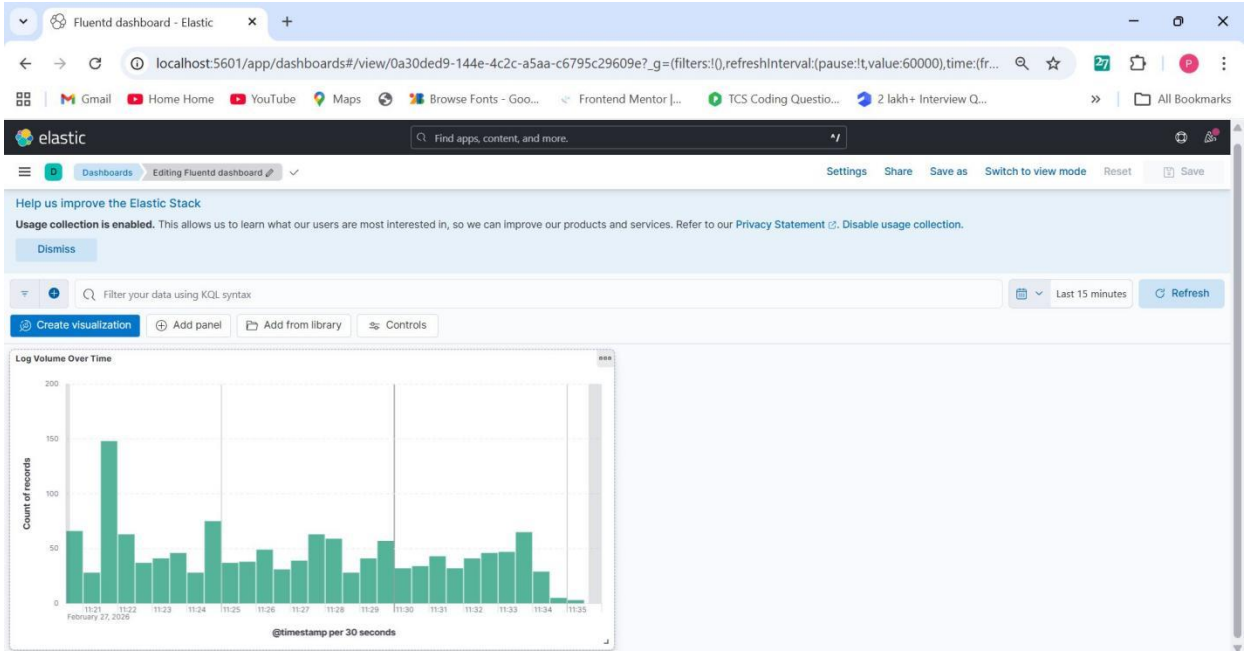

Figure 7.3 — Kibana Fluentd Dashboard: Log Volume Over Time bar chart